

FINGERPRINTING GPU WORKLOADS

Jiri Gogela

LPC/Dec/2025

Intro

Jiri Gogela

Affiliation: Trend Micro
Location: EU/Czechia/Prague
Security research: network, containers

Github: https://github.com/gogela
Linkedin: https://www.linkedin.com/in/jirigogela/

Problem:

“Can we detect type of GPU workload at the runtime using Falco?”

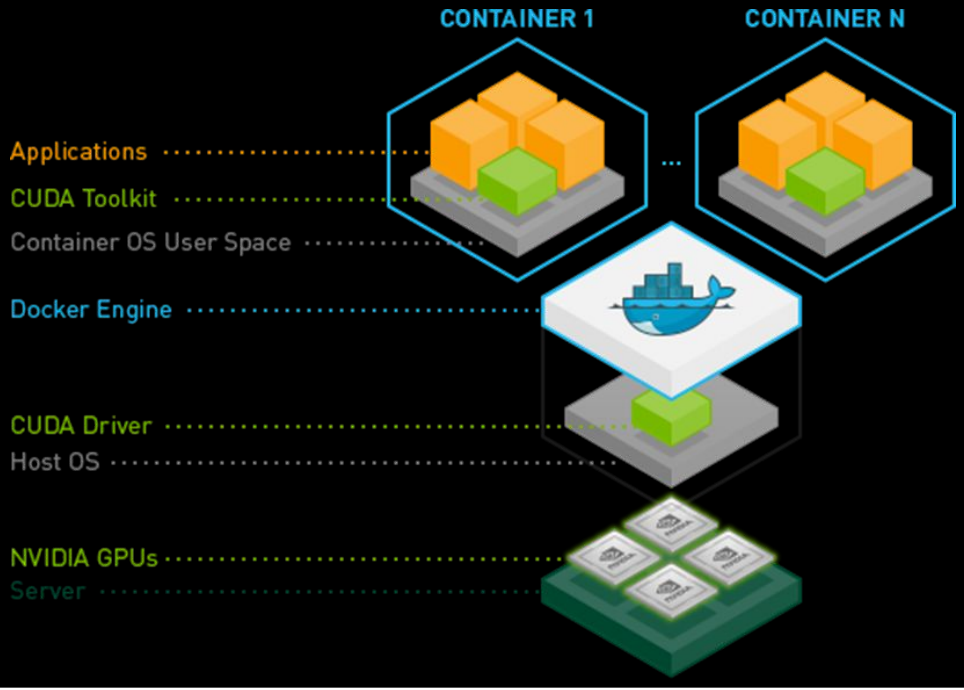


Syscalls: open, close, read, write, socket, bind, connect…….
(https://falco.org/docs/reference/rules/supported-events/)

NVIDIA container toolkit

Can we have a ‘Falco’ for GPU?

Just a few prev works like
“GPU Profiling with eBPF” by META



Kprobes - functions

A naive experiment

```
#cat /sys/kernel/debug/tracing/available_filter_functions | grep nvidia |wc -l
2843
```

how the 41 functions have been chosen…

Selected Functions/Symbols:

```
nvUvmInterfaceRegisterGpu, nvUvmInterfaceDupMemory, nvUvmInterfaceGetExternalAllocPtes,
nvUvmInterfaceBindChannelResources, nvUvmInterfaceChannelAllocate, nvUvmInterfaceFreeDupedHandle,
nvUvmInterfaceGetChannelResourcePtes, nvUvmInterfaceRetainChannel, nvUvmInterfaceSetPageDirectory,
nvUvmInterfaceTsgAllocate, nvUvmInterfaceAddressSpaceCreate, nvUvmInterfaceAddressSpaceDestroy,
nvUvmInterfaceChannelDestroy, nvUvmInterfaceDestroyAccessCntrInfo, nvUvmInterfaceDestroyFaultInfo,
nvUvmInterfaceDeviceCreate, nvUvmInterfaceDeviceDestroy, nvUvmInterfaceDupAddressSpace, nvUvmInterfaceGetEccInfo,
nvUvmInterfaceGetFbInfo, nvUvmInterfaceGetGpuInfo, nvUvmInterfaceGetPmaObject,
nvUvmInterfaceHasPendingNonReplayableFaults, nvUvmInterfaceInitAccessCntrInfo, nvUvmInterfaceInitFaultInfo,
nvUvmInterfaceMemoryAllocSys, nvUvmInterfaceMemoryCpuMap, nvUvmInterfaceMemoryCpuUnMap, nvUvmInterfaceMemoryFree,
nvUvmInterfaceOwnPageFaultIntr, nvUvmInterfacePmaAllocPages, nvUvmInterfacePmaFreePages,
nvUvmInterfacePmaRegisterEvictionCallbacks, nvUvmInterfacePmaUnregisterEvictionCallbacks, nvUvmInterfaceQueryCaps,
nvUvmInterfaceQueryCopyEnginesCaps, nvUvmInterfaceReleaseChannel, nvUvmInterfaceStopChannel, nvUvmInterfaceTsgDestroy,
nvUvmInterfaceUnregisterGpu, nvUvmInterfaceUnsetPageDirectory
```

Recording function calls

A trivial manual PoC

1. **Bpfrace example (probes.bt):**

```
kprobe:nvUvmInterfaceAddressSpaceCreate {
  printf("%u : ", nsecs(tai)/1000000);
  printf("Called nvUvmInterfaceAddressSpaceCreate PID %d %s | \n"  , pid, comm);}
kprobe:nvUvmInterfaceAddressSpaceDestroy {
  printf("%u : ", nsecs(tai)/1000000);
  printf("Called nvUvmInterfaceAddressSpaceDestroy PID %d %s | \n"  , pid, comm);}
```

etc...
for all 41 functions

2. **Collect data**

```
bpftrace -o out_testX.log probes.bt
+   Run the test container
```



```
Postprocess
data to numpy
dataframe
```

Initial PoC

Test workloads

- **TensorFlow benchmark/ ResNet training (cemizm/tf-benchmark-gpu)**
- **Llama inference (ggerganov/llama.cpp:full-cuda)**
- **XMRIG (cryptoandcoffee/akash-gpu-miner-xmrig-moneroocean-c11:19)**
- **Nvidia NIM (meta-llama3-8b-instruct:24.05.rc9)**

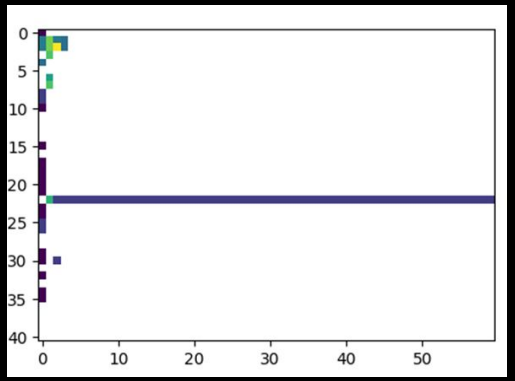
Fingerprint format

- **Matrix (numpy array)**
- **41 functions**
- **60 secs,**
- **values: number of fn calls during each 1 sec interval**

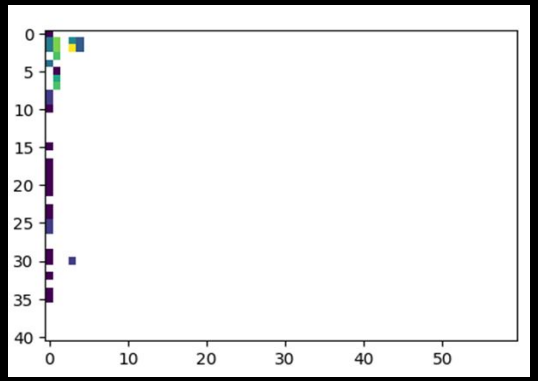
Test GPUs:

- **Nvidia A10 (AWS EC2 g5.xlarge)**
- **Nvidia V100 (AWS EC2 p3.xlarge)**

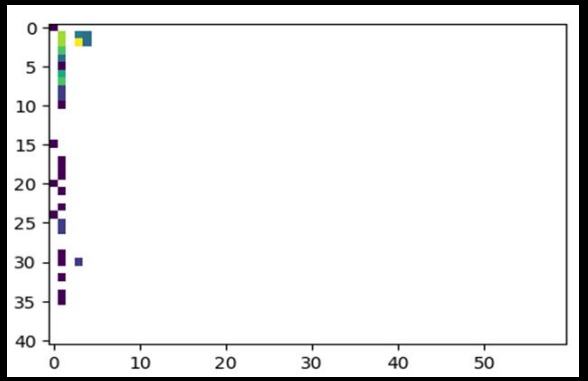
Fingerprints



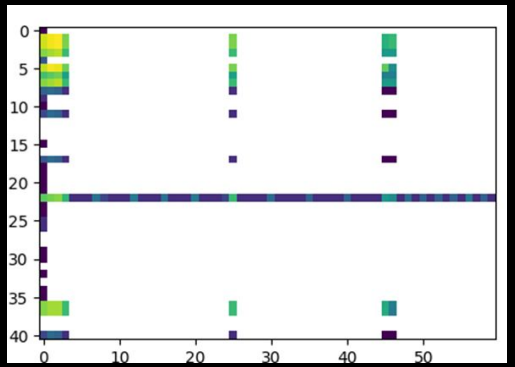
llama @V100 GPU



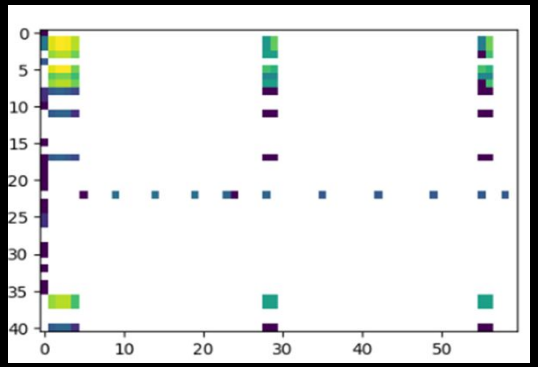
llama @A10 GPU



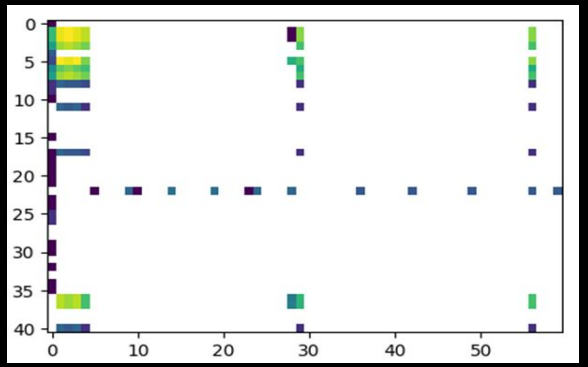
llama @A10 GPU Stressed



xmrig @V100 GPU



xmrig @A10 GPU



xmrig @A10 GPU Stressed

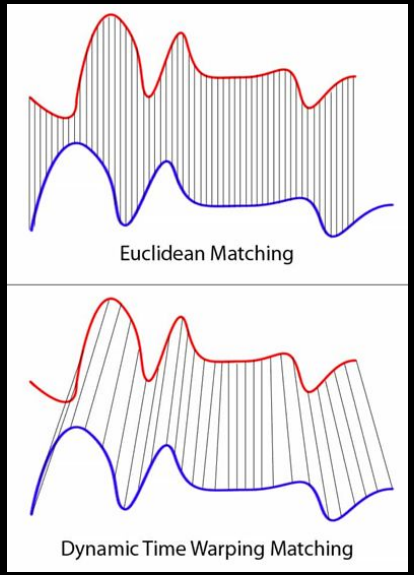
“Dilatation of time”

How to achieve comparable results for CPU under heavy load…

Metric

- **Compare time series of function calls**
- **Challenge: time shifts due to different performance or CPU load**
- **Dynamic Time Warping (DTW)**

```
def dtw_per_fn(arr1,arr2):
  res = np.zeros(len(arr1))
  for fn in range(len(arr1)):
    res[fn] = dtw(arr1[fn],arr2[fn])
  return np.linalg.norm(res)
```

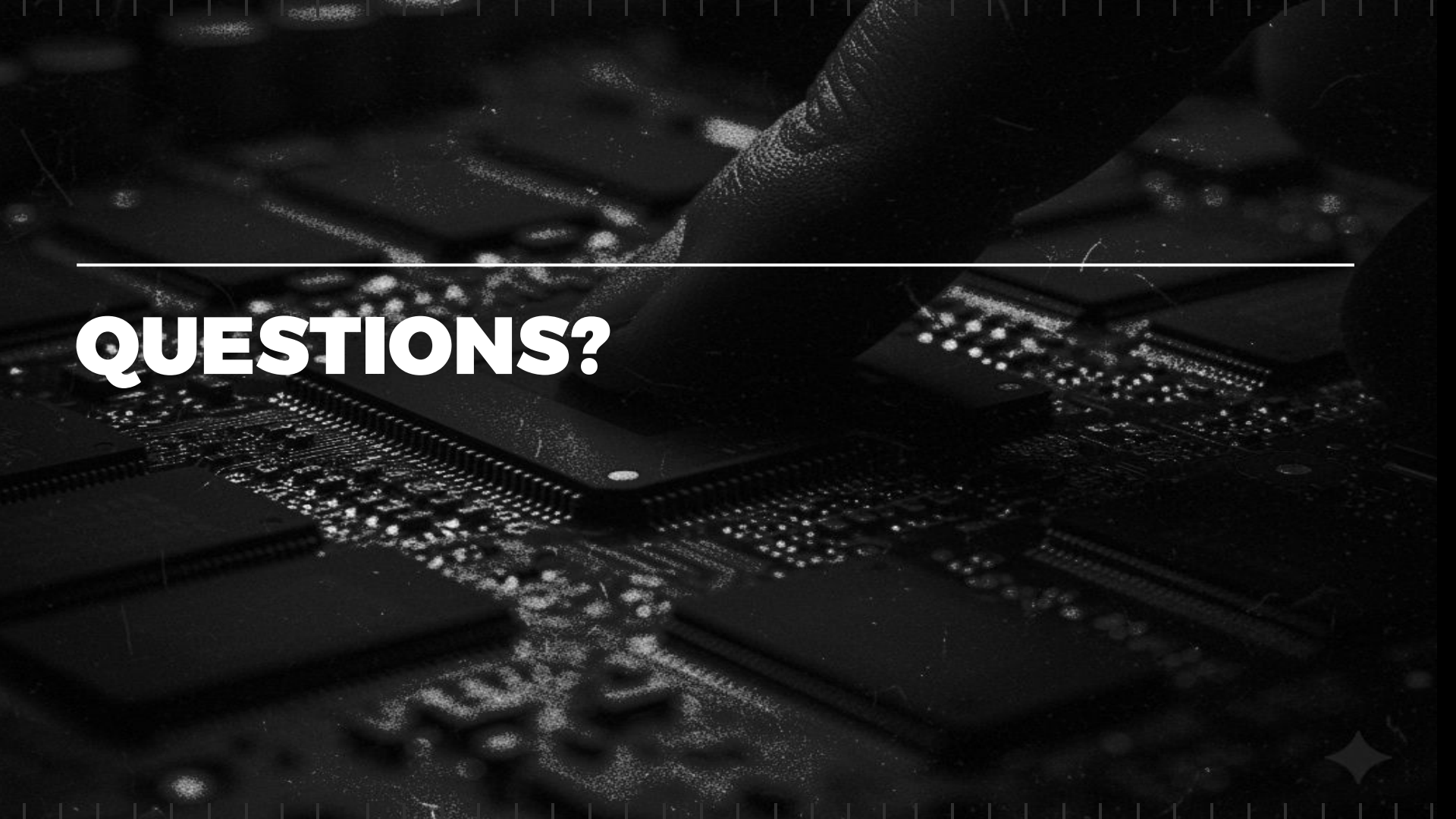


Fingerprints compared

WORKLOADS (@A10 GPU)	llama_cpu_stress1.txt	llama_cpu_stress2.txt	llama_cpu_stress3.txt	llama2.1.txt	llama2.2.txt	llama2.txt	nim2.txt	nim2-cpu-stress.txt	nim3.txt	tf_trivial.txt	tf_trivial2.txt	tf_trivial3.txt	tf2.txt	tf3.txt	tf4.txt	xmrig_cpu_stress1.txt	xmrig2.txt	xmrig3.txt
llama_cpu_stress1.txt	0.0	45.1	6.4	39.5	5.9	46.2	254.1	335.6	382.4	30.2	31.2	30.8	30.4	40.0	30.6	424.3	446.8	448.0
llama_cpu_stress2.txt	45.1	0.0	46.2	12.1	46.0	6.2	222.7	312.5	362.9	54.1	56.1	56.1	53.4	44.5	54.2	423.9	457.2	454.3
llama_cpu_stress3.txt	6.4	46.2	0.0	40.5	3.2	46.1	250.9	333.2	380.2	33.5	33.6	33.6	33.6	43.5	33.4	420.7	444.2	444.4
llama2.1.txt	39.5	12.1	40.5	0.0	40.4	12.2	228.2	316.1	366.1	49.9	52.0	52.0	49.2	41.4	50.0	426.3	461.5	456.0
llama2.2.txt	5.9	46.0	3.2	40.4	0.0	46.2	249.0	331.8	379.6	34.2	34.3	34.3	34.4	44.4	34.3	420.4	444.0	444.1
llama2.txt	46.2	6.2	46.1	12.2	46.2	0.0	222.8	312.6	362.9	62.4	63.8	64.7	61.7	53.1	62.4	433.0	457.7	458.8
nim2.txt	254.1	222.7	250.9	228.2	249.0	222.8	0.0	202.0	274.0	293.0	293.3	295.3	292.8	265.8	293.0	446.0	538.5	544.1
nim2-cpu-stress.txt	335.6	312.5	333.2	316.1	331.8	312.6	202.0	0.0	72.0	360.7	363.0	363.0	359.7	343.4	360.7	491.1	581.5	586.6
nim3.txt	382.4	362.9	380.2	366.1	379.6	362.9	274.0	72.0	0.0	404.4	405.7	405.7	403.8	392.5	404.3	529.0	607.3	612.6
tf_trivial.txt	30.2	54.1	33.5	49.9	34.2	62.4	293.0	360.7	404.4	0.0	5.3	5.5	4.7	27.7	5.3	438.8	454.2	458.7
tf_trivial2.txt	31.2	56.1	33.6	52.0	34.3	63.8	293.3	363.0	405.7	5.3	0.0	4.0	6.2	31.0	5.3	435.7	451.0	455.5
tf_trivial3.txt	30.8	56.1	33.6	52.0	34.3	64.7	295.3	363.0	405.7	5.5	4.0	0.0	4.5	30.9	3.7	436.2	451.5	456.0
tf2.txt	30.4	53.4	33.6	49.2	34.4	61.7	292.8	359.7	403.8	4.7	6.2	4.5	0.0	25.6	2.4	440.6	456.0	460.5
tf3.txt	40.0	44.5	43.5	41.4	44.4	53.1	265.8	343.4	392.5	27.7	31.0	30.9	25.6	0.0	27.3	454.3	472.4	475.8
tf4.txt	30.6	54.2	33.4	50.0	34.3	62.4	293.0	360.7	404.3	5.3	5.3	3.7	2.4	27.3	0.0	439.0	454.4	458.9
xmrig_cpu_stress1.txt	424.3	423.9	420.7	426.3	420.4	433.0	446.0	491.1	529.0	438.8	435.7	436.2	440.6	454.3	439.0	0.0	159.1	160.4
xmrig2.txt	446.8	457.2	444.2	461.5	444.0	457.7	538.5	581.5	607.3	454.2	451.0	451.5	456.0	472.4	454.4	159.1	0.0	94.8
xmrig3.txt	448.0	454.3	444.4	456.0	444.1	458.8	544.1	586.6	612.6	458.7	455.5	456.0	460.5	475.8	458.9	160.4	94.8	0.0

Applications

Application for detector of malicious payloads in GPU.
Similar approach might work for some other devices too.


QUESTIONS?